รายงานโครงงานในรายวิชา

การพัฒนาโปรแกรมประยุกต์สำหรับอุปกรณ์เคลื่อนที่

ภาคเรียนที่ 1 ปีการศึกษา 2569

ชื่อกลุ่ม/โปรเจ็ค

เกมฉันอยากเป็นผู้ชายคนนั้น At Home / Augustus Brutal

รายชื่อสมาชิก

รหัสนักศึกษา 016830641008-0 ชื่อ นายชัชชัย นาคแท้

รหัสนักศึกษา 016830641010-6 ชื่อ นายปุณยกร ธรรมวงษ์

แรงบันดาลใจ / ที่มาของโครงงาน

เกม "AUGUSTUS BRUTAL" ได้รับแรงบันดาลใจโดยตรงจากเกมระดับตำนานในแนว Masocore Platformer อย่าง "I Wanna Be The Guy" (IWBTG) และ "I Wanna Be The Guy: Remastered" ผสมผสานกับเสน่ห์ของคลิปสตรีมเมอร์ไทย (เช่น ช่อง Flook611 Gameplay) ที่สร้างความบันเทิงและเสียงหัวเราะผ่านการเผชิญหน้ากับกับดักสุดกวนที่คาดเดาไม่ได้

- ความท้าทายที่แตกต่าง: เกมแพลตฟอร์มเมอร์ดั้งเดิมมักเน้นการกระโดดข้ามสิ่งกีดขวางตามรูปแบบที่เป็นเส้นตรง แต่เกมแนว IWBTG เน้นการใช้ "จิตวิทยา" หลอกล่อผู้เล่น (Troll Traps) เช่น ผลไม้ที่ปกติควรตกลงล่างกลับพุ่งขึ้นฟ้า, หนามที่ดูเหมือนของประดับฉากกลับพุ่งเข้าใส่ผู้เล่นในระยะประชิด (Point-Blank Range)
- การเรียนรู้ผ่านความผิดพลาด (Trial and Error): หัวใจหลักของเกมแนวนี้คือการที่ผู้เล่นต้อง "ตายซ้ำแล้วซ้ำเล่า" เพื่อจดจำตำแหน่งของกับดัก และพัฒนาทักษะการบังคับอย่างแม่นยำ
- การพิสูจน์ศักยภาพของ Flutter & Flame: ผู้จัดทำต้องการพิสูจน์ว่า Flutter Framework ร่วมกับ Flame Engine สามารถพัฒนาเกมแอ็กชัน 2D ที่ต้องการการคำนวณฟิสิกส์ที่แม่นยำระดับพิกเซล (Sub-pixel Precision), การตอบสนองอินพุตที่ฉับไว (Zero Latency), และการจัดการ Collision ที่มีประสิทธิภาพสูงได้ไม่แพ้เกมเอนจินเฉพาะทาง

อธิบายการทำงานของโปรแกรม

ตัวเกมถูกออกแบบด้วยสถาปัตยกรรมแบบ Component-Based บน Flame Engine มีระบบการทำงานสำคัญดังนี้:

1. **ระบบฟิสิกส์และการควบคุมตัวละคร (The Kid Physics):**
    - **Double Jump & Variable Jump:** ตัวละครสามารถกระโดดสองจังหวะกลางอากาศได้อย่างอิสระ และควบคุมความสูงของการกระโดดได้ตามระยะเวลาที่กดปุ่มกระโดดค้างไว้
    - **Coyote Time & Jump Buffering:** มีระบบหน่วงเวลาเสี้ยววินาทีให้ผู้เล่นกดกระโดดได้แม้เท้าจะเพิ่งหลุดจากขอบแท่น และจำคำสั่งกระโดดล่วงหน้าก่อนตกถึงพื้น ช่วยให้การควบคุมลื่นไหลไม่ติดขัด
    - **Instant Zero-Delay Respawn:** เมื่อผู้เล่นตาย เลือดจะพุ่งกระจายอย่างสะใจ และสามารถกดปุ่ม **R** เพื่อเกิดใหม่ ณ จุดเกิดได้ทันทีในเวลา 0 วินาที เพื่อไม่ให้เสียจังหวะและอารมณ์ในการเล่น
2. **ระบบกับดักและเซนเซอร์ตรวจจับ (Troll Traps & Trigger Zones):**
    - **Point-Blank Proximity Triggers:** กับดักจะไม่ทำงานหากผู้เล่นอยู่ห่าง แต่จะกระตุ้นการทำงานทันทีเมื่อผู้เล่นก้าวเข้าสู่ระยะประชิดที่ไม่อาจตั้งตัวได้ทัน
    - **Anti-Gravity Apples & Sliding Spikes:** แอปเปิ้ลพุ่งสวนแรงโน้มถ่วง, หนามสไลด์ไป-กลับบนแท่น, และหนามซ่อนตัวใต้พื้นแท่นที่จะแทงขึ้นมาเมื่อเหยียบ
    - **Off-Screen Rushers:** กับดักหนามที่ซ่อนอยู่นอกจอแล้วพุ่งกวาดผ่านฉากด้วยความเร็วสูงกว่า 1,100 px/s

ระบบการจำลองเลือด (Procedural Raycast Blood Physics):

• **Raycast Surface Sticking:** เมื่อตัวละครตาย

ระบบจะสร้างเม็ดเลือดพิกเซลเรโทรจำนวนมากพุ่งกระจายตามแรงระเบิดแบบสุ่ม โดยเม็ดเลือดจะใช้ระบบ Raycast สแกนพื้นผิว

และตกลงไปแปะติดแน่นบนแพลตฟอร์มหรือขยับตามแพลตฟอร์มที่เคลื่อนที่ได้อย่างสมจริง

ระบบตรวจสอบฮิตบ็อกซ์แบบเรียลไทม์ (Hazard Debug Hitbox Viewer):

• **Visual Debugging:** ผู้พัฒนาและผู้เล่นสามารถกดปุ่ม H บนคีย์บอร์ด หรือแตะไอคอนรูปตาบนแถบ HUD ด้านบน เพื่อเปิด/ปิดโหมด Debug Hitbox โดยระบบจะวาดโครงร่างเรขาคณิต (สามเหลี่ยมสีเขียวนีออนสำหรับหนาม, วงกลมสำหรับผลไม้, และกล่องสี่เหลี่ยมสีฟ้าสว่างสำหรับ Trigger Zone) ทับบนหน้าจอแบบเรียลไทม์ ช่วยให้เห็นตำแหน่งที่แท้จริงของกับดักที่ซ่อนอยู่

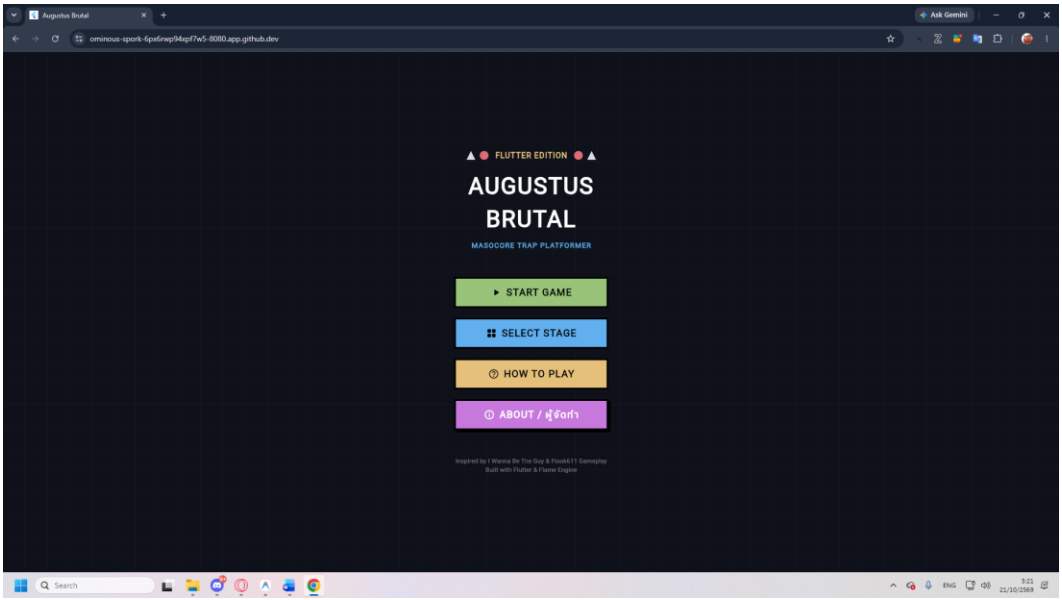
การรองรับข้ามแพลตฟอร์ม (Cross-Platform Controls):

• **Cross-Platform Support:** รองรับการเล่นบน PC และ Web ผ่านปุ่มคีย์บอร์ดมาตรฐาน (A, D, Space, Shift, W, R, H) และรองรับการเล่นบน Mobile (สมาร์ตโฟน) ผ่านปุ่มจำลองบนหน้าจอสัมผัส (Virtual D-Pad, ปุ่ม JUMP, ปุ่ม R เกิดใหม่ทันที) พร้อมระบบป้องกันอินพุตค้างเมื่อตัวละครเกิดใหม่

ภาพประกอบ (Screenshots & Showcase)

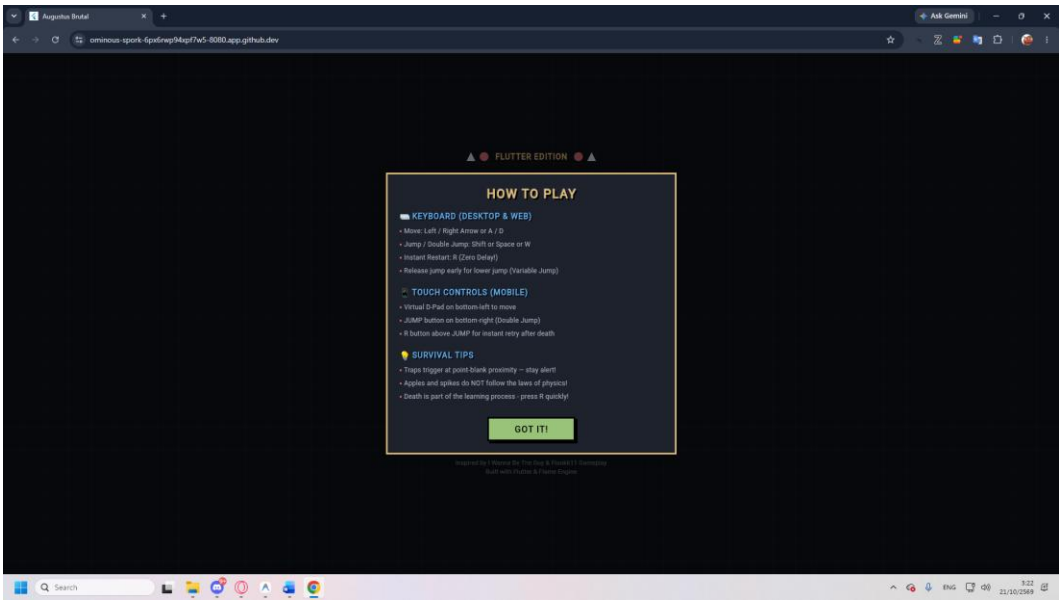
ภาพถ่ายหน้าจอแสดงผลการทำงานจริงของเกม AUGUSTUS BRUTAL ครอบคลุมทั้งหน้าจอเมนูหลัก, หน้าต่างแนะนำวิธีเล่น, หน้าจอข้อมูลผู้จัดทำ และการเผชิญหน้ากับกับดักในแต่ละด่าน:

ภาพที่ 1: หน้าจอเมนูหลัก (Main Menu Screen)



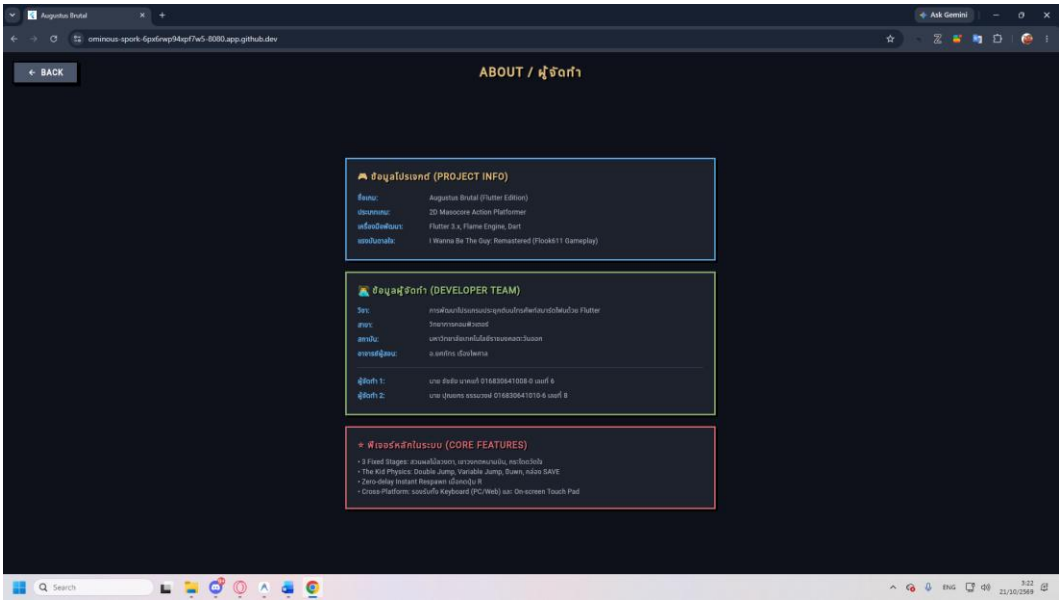
คำอธิบาย: แสดงชื่อเกม "AUGUSTUS BRUTAL" สไตล์เรโทรพิกเซลแบบ Flat คมชัด พร้อมปุ่มตัวเลือก START GAME, SELECT STAGE, HOW TO PLAY, และ ABOUT / ผู้จัดทำ

ภาพที่ 2: หน้าต่างวิธีเล่น (How to Play Screen)



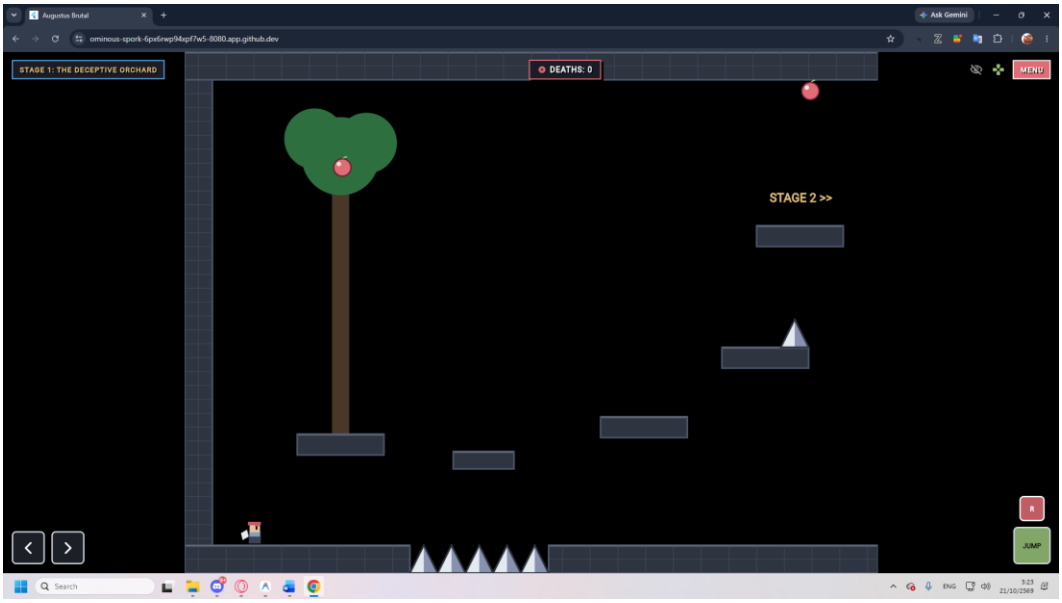
คำอธิบาย: แสดงคำแนะนำการควบคุมทั้ง Keyboard สำหรับ PC/Web (A/D, Space/Shift, R, H) และระบบ Virtual Controls สำหรับหน้าจอสัมผัสบน Mobile พร้อมคำแนะนำปรัชญาการเล่นสไตล์ Masocore

ภาพที่ 3: หน้าจอข้อมูลผู้จัดทำ (About Screen)



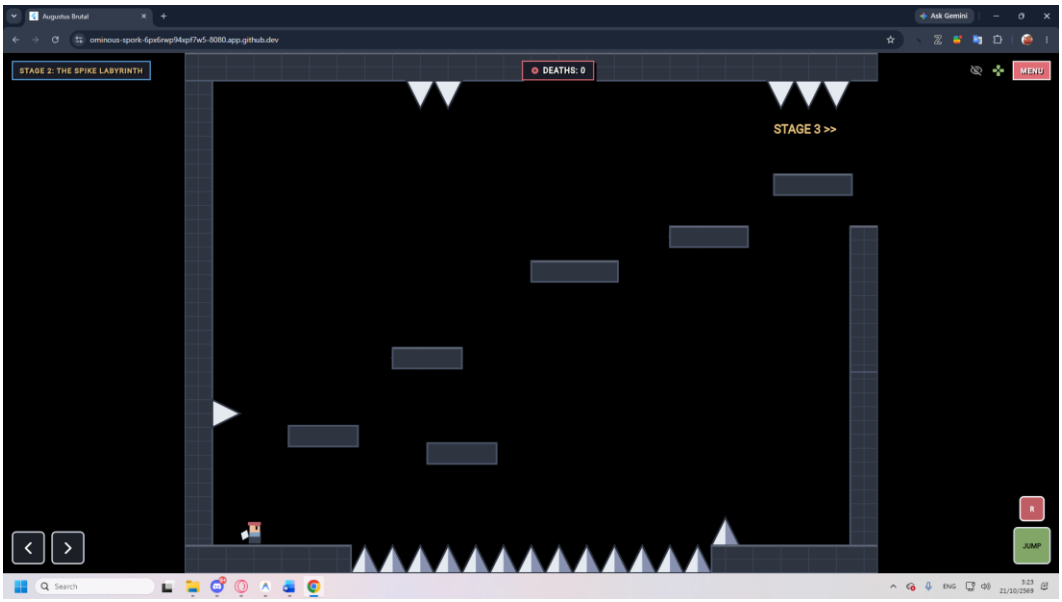
คำอธิบาย: แสดงข้อมูลรายวิชา สาขาวิชา คณะ สถาบัน อาจารย์ผู้สอน และรายชื่อสมาชิกกลุ่ม Developer Team (นายชัชชัย นาคแท้ และ นายปุณยกร ธรรมวงษ์) พร้อมปุ่มย้อนกลับลูกศรชี้ซ้าย

ภาพที่ 4: ด่านที่ 1 - สวนผลไม้ลวงตา (Stage 1: Orchard of Deception)



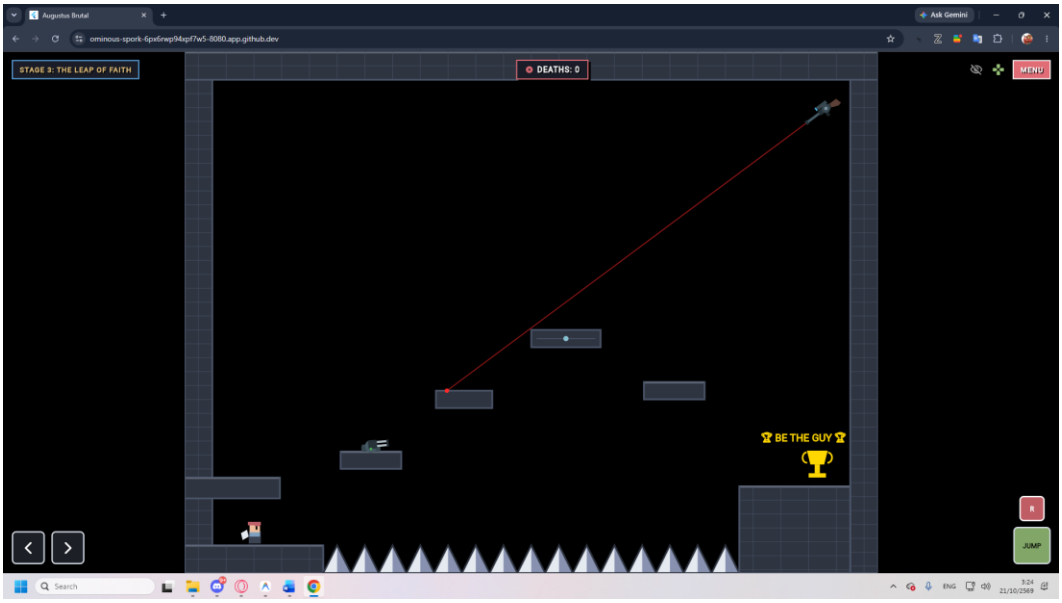
คำอธิบาย: แสดงสภาพแวดล้อมด่านที่ 1 กับดักแอปเปิ้ลพุ่งสวนแรงโน้มถ่วง, หนามสไลด์บนแท่นหิน, และระบบแถบ HUD ด้านบนพร้อมปุ่มเปิด/ปิด Debug Hitbox และ Virtual Controls

ภาพที่ 5: ด่านที่ 2 - เขาวงกตหนามบิน (Stage 2: Spike Maze)



คำอธิบาย: แสดงความท้าทายของด่านที่ 2 กับดักหนามพุ่งทะลุฉากด้วยความเร็วสูง (Rushers) และการควบคุมจังหวะการกระโดด Double Jump หลบหนามกลางอากาศในช่องทางแคบ

ภาพที่ 6: ด่านที่ 3 - กระโดดวัดใจและป้อมปืนกล (Stage 3: Leap of Faith & Turret Arena)



คำอธิบาย: แสดงระบบป้อมปืนกลกวาดองศาการเล็ง (Turret Hazard), เลเซอร์สไนเปอร์ล็อกเป้า, และการกระโดดวัดใจข้ามเหวลึกผ่านแท่นกระโดดที่ต้องการความแม่นยำระดับพิกเซล

เครื่องมือที่ใช้ (App Inventor / Flutter)

โครงงานนี้พัฒนาด้วย Flutter Framework โดยมีรายละเอียดเครื่องมือและเทคโนโลยีที่ใช้ดังนี้:

- **ภาษาที่ใช้พัฒนา: Dart 3.x -** ภาษาโปรแกรมเชิงวัตถุประเภท Modern Strongly-Typed สำหรับพัฒนาตรรกะเกมและระบบฟิสิกส์
- **เฟรมเวิร์กหลัก: Flutter 3.x -** เฟรมเวิร์กหลักสำหรับการสร้าง User Interface ข้ามแพลตฟอร์ม (Cross-Platform) รองรับทั้ง Android, iOS, Web และ Windows
- **เกมเอนจิน: Flame Engine (v1.x) -** เกมเอนจิน 2D ประสิทธิภาพสูงบน Flutter โดยใช้จัดการ Game Loop (Update & Render), ระบบตรวจจับการชน (Collision Detection), และ Canvas API สำหรับการวาดกราฟิกพิกเซล
- **การจัดการเวอร์ชัน (Version Control): Git & GitHub -** ระบบควบคุมเวอร์ชันซอร์สโค้ด และจัดเก็บบนคลาวด์สำหรับทำงานร่วมกันในทีม
- **สภาพแวดล้อมการพัฒนา (IDE): Visual Studio Code / Google Antigravity -** โปรแกรมแก้ไขโค้ดและสภาพแวดล้อมการพัฒนาแอปพลิเคชัน

• **เครื่องมือทดสอบและตรวจสอบคุณภาพ: flutter test & flutter analyze -** ใช้คำสั่ง flutter test ในการทำ Automated Unit Testing สำหรับฟิสิกส์และกับดัก และคำสั่ง flutter analyze ในการตรวจสอบคุณภาพและมาตรฐานของโค้ด

ลิงค์ repo และลิงค์วิดีโอ

• **ลิงก์ GitHub Repository:** https://github.com/ChatchaiNaktae/Augustus.git

• **ลิงก์วิดีโอสาธิตการเล่น (Gameplay Video):** [ใส่ลิงก์วิดีโอสาธิตการเล่น YouTube / Google Drive ที่นี่]

Prompt AI ที่ใช้ (AI Prompts & Collaboration)

ในการพัฒนาโครงงานนี้ ได้ประยุกต์ใช้ AI Assistant เพื่อร่วมพัฒนาแบบ Pair Programming ในการช่วยวิเคราะห์ฟิสิกส์ ออกแบบระบบ ตรวจสอบความถูกต้องของโค้ด และแก้ไขบั๊กที่ซับซ้อน โดยมีตัวอย่างชุดคำสั่ง (Prompts) สำคัญดังนี้:

1. การปรับความเร็วกับดักและแก้บั๊กฟิสิกส์ (Physics & Trap Tuning):

"ตอนนี้พวกกับดัก ผลไม้ และหนาม มันยังช้าอยู่อ่ะ ช่วยแก้ให้มันเร็วขึ้นในระดับ Masocore แท้ และแก้บั๊กการชนแพลตฟอร์มที่ทำให้ตัวละครเกิดอาการวาปหรือกระตุกเมื่อกระโดดเฉียงเข้าหาขอบแพลตฟอร์ม โดยให้ /plan แผนงานออกมาก่อน"

2. การพัฒนาระบบตรวจสอบฮิตบ็อกซ์ (Hazard Debug Hitbox System):

"เพิ่ม Debug Hitbox ตอนกดปุ่มบน Keyboard (ปุ่ม H) และปุ่ม UI รูปตาบนหน้าจอ โดยพอกดแล้วให้แสดง Hitbox เฉพาะกับดักต่าง ๆ ในแมพ (หนามและผลไม้) รวมถึงกล่อง Trigger Zone ด้วย"

3. การออกแบบและแก้ปัญหาทางเรขาคณิตของ Trigger Zone (Overlap Fixing):

"Trigger Zone แนวนอนบนแผ่นหินข้ามเหวมันซ้อนกับ Trigger Zone แนวตั้ง ช่วยแยก Trigger Zone แนวนอนออกเป็น 2 ท่อนขนาบข้างเพื่อไม่ให้พิกัดซ้อนทับกัน และปรับลดความเร็วหนามสไลด์ลง 75% ให้มีจังหวะกระโดดหลบได้อย่างสมเหตุสมผล"

4. การปรับปรุง UI และข้อมูลผู้จัดทำ (UI Polish & Developer Credits):

"ในหน้า Main Menu ให้เปลี่ยนชื่อเป็น AUGUSTUS BRUTAL ตัวพิมพ์ใหญ่ทั้งหมด, ปรับปุ่ม BACK ให้เหลือเฉพาะลูกศรชี้ซ้าย, เอา Drop Shadow ของตัวหนังสือออกให้เป็นแบบ Flat คมชัด และเพิ่มรายชื่อสมาชิกกลุ่มทั้ง 2 คนลงในหน้า About"

ปัญหาที่เจอ และวิธีแก้ (Problems & Solutions)

ในระหว่างการพัฒนาโครงงาน พบปัญหาทางเทคนิคและได้ดำเนินการแก้ไขอย่างเป็นระบบดังนี้:

1. ปัญหาการชนขอบแพลตฟอร์มแล้วตัวละครกระตุกหรือวาป (Corner Snagging & Seam Stuttering)

• **สาเหตุของปัญหา:** การคำนวณการเคลื่อนที่ของตัวละครในแกน X และ Y พร้อมกันในขั้นตอนเดียว ทำให้เมื่อตัวละครกระโดดเฉียงเข้าหามุมกล่อง กลไก Overlap Resolution ตรวจพบการทับซ้อนและผลักตัวละครลงไปด้านล่างอย่างผิดธรรมชาติ หรือติดขัดตามรอยต่อระหว่างบล็อกที่วางเรียงติดกัน

• **แนวทางการแก้ไข:** ปรับสถาปัตยกรรมการชนใน Player._moveWithCollisions ให้แยกการประมวลผลแกน X ก่อน แล้วตามด้วยแกน Y แบบ Sub-stepping พร้อมเพิ่มขอบเขตตรวจสอบมุมกล่อง ทำให้ตัวละครสามารถสไลด์เลียบผนังและก้าวข้ามรอยต่อระหว่างบล็อกได้อย่างลื่นไหล ไม่เกิดอาการกระตุกหรือดีดลงล่าง

2. ปัญหาวัตถุความเร็วสูงทะลุตัวละคร (Tunneling Effect)

• **สาเหตุของปัญหา:** กับดักหนามพุ่งและกระสุนสไนเปอร์เคลื่อนที่ด้วยความเร็วสูงมาก (1,100 - 4,500 px/s) ในกรณีที่เฟรมเรตตก (dt เพิ่มขึ้น) ระยะทางการเคลื่อนที่ใน 1 เฟรมจะกว้างกว่าขนาดตัวละคร ส่งผลให้ฮิตบ็อกซ์ข้ามตำแหน่งตัวละครไปโดยไม่เกิดการชน (Tunneling)

• **แนวทางการแก้ไข:** สร้างกลไก Anti-Tunneling Detection ด้วยการทำ Continuous Sweep Detection และ Raycasting ลากเส้นตรวจสอบระหว่างตำแหน่งในเฟรมก่อนหน้าและตำแหน่งปัจจุบัน ทำให้ตรวจจับการชนได้แม่นยำ 100% แม้จะเกิดอาการแล็กชั่วขณะ

3. ปัญหา Trigger Zone ซ้อนทับกันทำให้เกิดการยิงกับดักซ้ำซ้อน

• **สาเหตุของปัญหา:** บนแผ่นหินกลางเหวด่านที่ 1 มี Trigger Zone สองชุด คือ หนามพุ่งแนวนอนข้ามแมพ และหนามจรวดพุ่งแนวตั้งจากใต้เหว เมื่อกล่องแนวนอนพาดผ่านทั้งแผ่นหิน
ทำให้จุดกึ่งกลางทับซ้อนกับกล่องแนวตั้ง เมื่อผู้เล่นเหยียบจะกระตุ้นกับดักทั้งสองพร้อมกันโดยไม่ตั้งใจ

• **แนวทางการแก้ไข:** ใช้ระบบ Debug Hitbox ตรวจสอบพิกัด และทำการแยก Trigger Zone แนวนอนออกเป็น 2 กล่องย่อย (ซ้าย-ขวา) ขนาบข้างกล่องแนวตั้ง ทำให้ไม่มีพื้นที่ทับซ้อนเชิงเรขาคณิต และกับดักทำงานแยกกันตามจุดที่ผู้เล่นสัมผัสอย่างแม่นยำ

4. ปัญหาความเร็วของกับดักหนามสไลด์เร็วเกินกว่าปฏิกิริยาของมนุษย์

• **สาเหตุของปัญหา:** หนามสไลด์บนแท่นบันไดวิ่งด้วยความเร็ว 450 - 480 px/s บนระยะทางสไลด์เพียง 68 พิกเซล ส่งผลให้หนามเคลื่อนที่ถึงตัวผู้เล่นภายในเวลาเพียง 0.15 วินาที ซึ่งเร็วกว่า Human Reaction Time ทำให้ผู้เล่นไม่สามารถตอบสนองได้ทัน

• **แนวทางการแก้ไข:** ปรับลดความเร็วลง 75% เหลือ 115 - 120 px/s ส่งผลให้หนามใช้เวลาเคลื่อนที่ราว 0.55 - 0.59 วินาที ซึ่งเป็นช่วงเวลาที่ท้าทายแต่เปิดโอกาสให้ผู้เล่นสามารถกระโดดหลบ (Dodge & Jump) ได้อย่างมีชั้นเชิง

5. ปัญหาตัวหนังสือในปุ่มเมนูเบลอและมีเงาซ้อนจากการใส่ Drop Shadow

• **สาเหตุของปัญหา:** สไตล์ข้อความเดิมของ RetroTheme.pixelText มีการกำหนดค่าเงา Shadow(offset: Offset(2, 2), color: Colors.black) ไว้ เมื่อนำมาใช้กับตัวอักษรสีดำบนปุ่มกด
ทำให้เกิดเงาสีดำซ้อนกันจนตัวอักษรหนาผิดรูปและอ่านยาก

• **แนวทางการแก้ไข:** เพิ่มคุณสมบัติ hasTextShadow ให้กับ RetroButton และฟังก์ชันสร้างสไตล์ข้อความ โดยปิด Drop Shadow ของตัวหนังสือในหน้าเมนูหลักทั้งหมด คงไว้เฉพาะสไตล์ Flat Pixel Art ที่คมชัด สบายตา และอ่านง่ายในทุกขนาดหน้าจอ